

## Lista 4

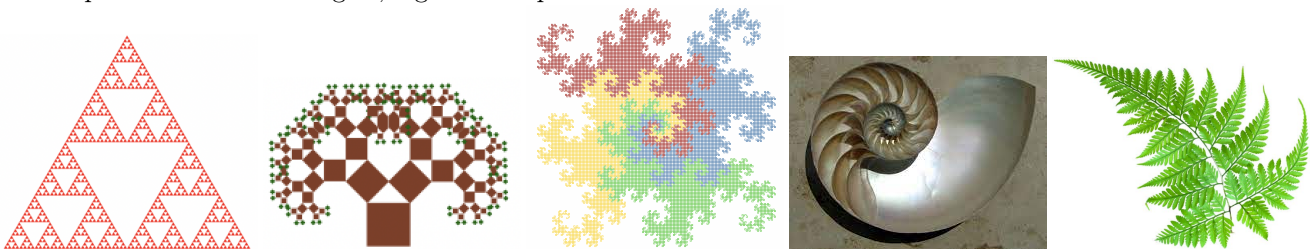
O exercício pode ser feito em DUPLAS. Para evitar que o arquivo fique com tamanho muito grande, não precisa colocar exemplos ou testes para as funções que geram desenhos. Ao invés dos testes deixe apenas as chamadas das funções.

### Instruções

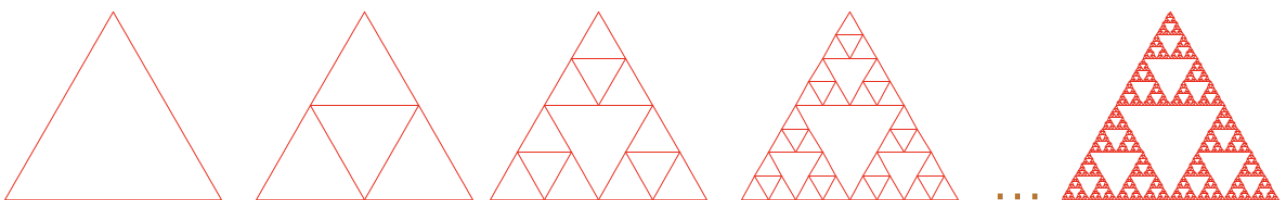
- USE OS NOMES DE TIPOS DE DADOS E FUNÇÕES DEFINIDOS NAS QUESTÕES.
- Use o **template da solução disponível no Moodle**.
- TODAS as funções, inclusive as auxiliares, devem ter documentação completa: contrato, objetivo, e pelo menos 2 exemplos e 2 testes (só não precisa incluir exemplos/testes nas funções que geram imagens). Se quiser, pode usar os testes como exemplos, mas neste caso isso deve ser indicado (leia o texto sobre exemplos e testes).
- Siga as instruções para a realização de exercícios disponível no Moodle (no item *Informações gerais*).
- **Para todas as funções recursivas você deve incluir o modelo da solução. Esse modelo, em forma de comentários que podem ser colocados antes ou permeados ao código.**

O template já possui o início de várias funções, incluindo documentação e testes em algumas questões, é só completar. Não modifiquem as definições do template!

Nesta lista vamos trabalhar com desenhos que são gerados de forma recursiva, os desenhos mais conhecidos deste tipo são os *fractais*. Fractais são imagens que possuem um padrão na sua construção, no qual as partes refletem o padrão do todo. A seguir, alguns exemplos de fractais.



A imagem mais à esquerda da lista se chama *triângulo de Sierpinski*. Vamos começar desenhando esta imagem. Esta imagem é gerada por sucessivas divisões de um triângulo em 3 triângulos, como mostra a imagem a seguir (os *triângulos invertidos* que aparecem na imagem não são desenhados, eles são o *espaço vazio* que se forma a partir dos desenhos dos 3 triângulos em cada etapa).



O template da lista de exercícios tem a definição da função **sierpinski** que, dados o tamanho do lado e uma cor, desenha um triângulo de Sierpinski desta cor cujo lado do triângulo externo é o lado passado como argumento. Nesta lista usaremos cores no formato RGB (ver definição do tipo **Color** no modelo da solução). Aplique esta função a vários argumentos diferentes para gerar triângulos de Sierpinski. Esta função pára de desenhar quando o lado do triângulo é menor ou igual a 5 (este é o caso trivial deste programa). Modifique o caso trivial do programa para outros valores para ver o que acontece (no final, deixe o caso trivial original para resolver as questões a seguir).

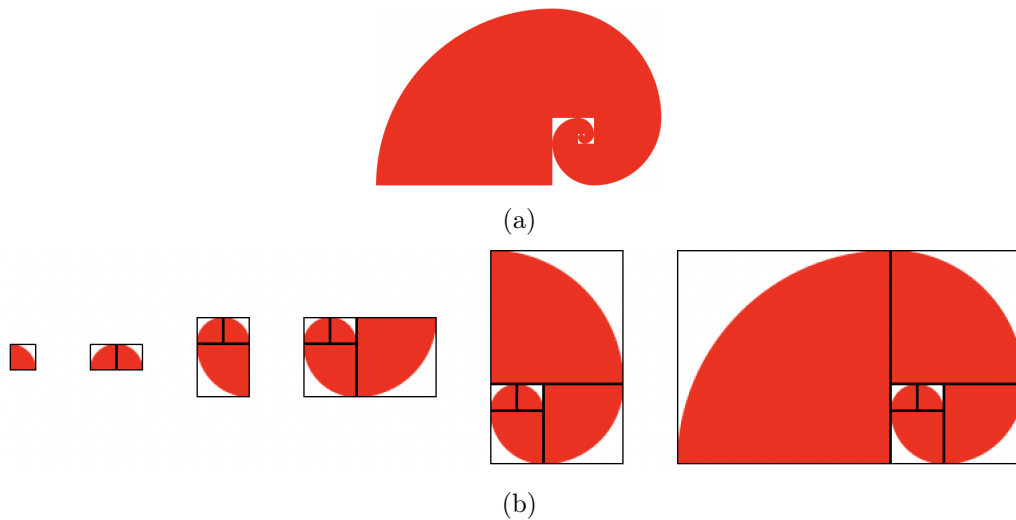
1. Podemos usar definições de nomes locais para facilitar a construção de soluções, bem como para torná-las mais eficientes. Um exemplo é a definição da função **sierpinski**, na qual definimos o nome **TRIANGULO** localmente para usa-lo na construção do desenho que queríamos montar. No entanto, podemos construir uma função que retorna o mesmo resultado sem utilizar definições locais. Construa a função **sierpinski-sem-def-local**, que deve ter exatamente o mesmo comportamento da função **sierpinski**, mas não pode ter definições locais.

2. A série de Fibonacci pode ser obtida usando a função matemática a seguir:

$$fibonacci(n) = \begin{cases} 0 & , \text{ se } n = 0 \\ 1 & , \text{ se } n = 1 \\ fibonacci(n-1) + fibonacci(n-2) & , \text{ caso contrário} \end{cases}$$

Construa a função **fibonacci** que, dado um número, gera o elemento correspondente da série de Fibonacci. Assuma que o número passado será um natural maior ou igual a zero.

Desenhando setores de círculos com ângulos retos tendo como raio no valor dos números da série de Fibonacci, encontra-se uma espiral perfeita, presente em muitos elementos da natureza. O formato de algumas conchas, como o náutilus, segue o padrão da série de Fibonacci. Desenvolva a função **espiral** que, dado um número e uma cor (no padrão RGB), gera uma imagem conforme a ilustração abaixo (a), ela imagem foi gerada com setores até o décimo elemento da série de Fibonacci. A figura (b) enfatiza como os setores foram desenhados (mostra os seis primeiros passos). Para seu desenho, use o elemento da série de Fibonacci como raio para desenhar o setor correspondente (os primeiros setores ficarão bem pequenos, mas como a série aumenta muito rápido, é melhor não multiplicar o raio por algum fator).

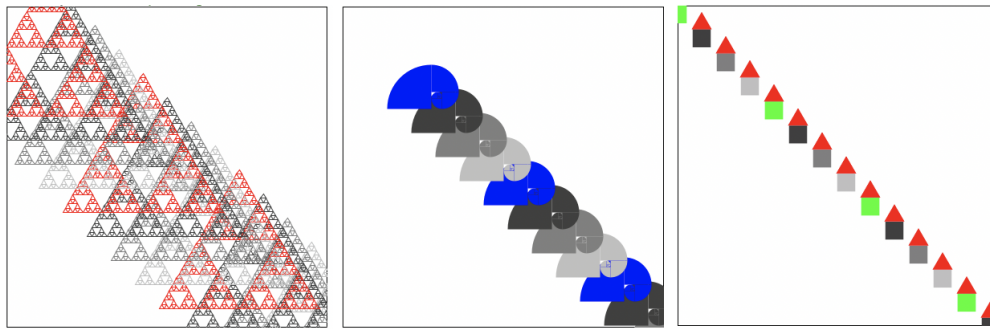


3. A função **sierpinski** desenha um triângulo de Sierpinski, dados o tamanho do lado e a cor. Nos próximos exercícios vamos desenharmos figuras em cenas. Para isso, é necessário informar a posição na qual a figura deve ser desenhada (coordenadas x e y). O tipo **Figura** está definido no modelo, possuindo 4 argumentos: a coordenada x da figura, a coordenada y da figura, o fator de escala para desenharmos a figura e o a cor da figura (tipo **Color**).

Construa a função **desenha-sierpinski** para gerar um triângulo de Sierpinski recebendo um argumento do tipo **Figura**. A função deve desenharmos um triângulo de Sierpinski com as especificações dadas na figura passada como argumento. Note que a definição da figura não tem o tamanho do lado, portanto você deve usar um tamanho fixo: use a constante **LADO-TRIANGULO**. O fator de escala da figura deve ser usado como argumento para a função **scale** gerando uma figura na escala definida pela figura. Use também a função **sierpinski** para desenharmos o triângulo de Sierpinski. Construa, também, a função **desenha-espiral**, que dado um argumento do tipo **Figura**, desenha uma espiral como definido na questão 2, usando a constante **NUM-FIBO** como entrada para a função **espiral**.

4. Agora vamos construir uma função mais genérica que gera uma cena com várias figuras. Para desenharmos as figuras, o programa vai receber a função que desenha a figura. A ideia é desenharmos esta figura em uma cena, modificar algo na figura (coordenadas x e/ou y, cor, escala), colocar esta imagem na cena, e assim por diante, até que alguma condição de término seja alcançada.

Desenvolva o programa **cena-com-figuras**, que desenha várias figuras em uma cena, variando a sua cor, escala e/ou sua localização. O programa recebe a função que desenha a figura e uma figura (tipo **Figura**) contendo a posição inicial, o escala inicial e a cor inicial. O programa pode variar a cor, escala e posição da figura, até que um critério de fim seja encontrado. Defina como quiser qual como a próxima figura a ser desenhada (se e como muda a cor, escala, posição) e qual o critério de fim. Alguns exemplos podem ser vistos na figura a seguir. Nestas figuras foi usada uma cena com largura e altura de 400 pontos, o critério de fim foi o centro da figura estar fora dos limites da cena, e a cada passo foram modificadas a cor da figura (usando a função **ciclo** definida no modelo) e a posição (aumentando 30 unidades tanto na coordenada x quanto na y da figura).



5. Generalize a função do exercício anterior:

- Construa 3 funções para testar critérios diferentes de fim.
- Construa 2 funções para modificar valores RGB: estas são funções que, dado um número (entre 0 e 255), geram um outro número (entre 0 e 255). As funções `ciclo` e `aleatorio` do modelo são exemplos. Essas funções podem ser passadas como argumento para a função `fun-muda-cor` para gerar uma função que modifica cores.
- Construa, também, diferentes funções de alteração de figuras (pelo menos 3 funções). Pelo menos uma das funções de alteração deve variar também a cor. Para gerar um movimento aleatório, pode-se usar a função `random`, que recebe um número natural e gera um número natural entre 0 e este número como resultado.
- Desenvolva a função de `cena-com-figuras-gen`, que recebe uma figura inicial, uma função de desenhar figura, uma função que testa um critério de fim e uma função de alteração de figuras, nesta ordem, e gera uma cena com várias figuras (como as da questão anterior). Use as funções dos itens (a), (b) e (c) para gerar exemplos diferentes para a função `cena-com-figuras-gen`. Construa pelo menos 4 chamadas diferentes da sua função. A seguir, exemplos de chamadas e os respectivos resultados – note que, com excessão dos valores do tipo `Figura` todos outros argumentos da função `cena-com-figuras-gen` são funções, que devem ser definidas para poder executar essas chamadas:

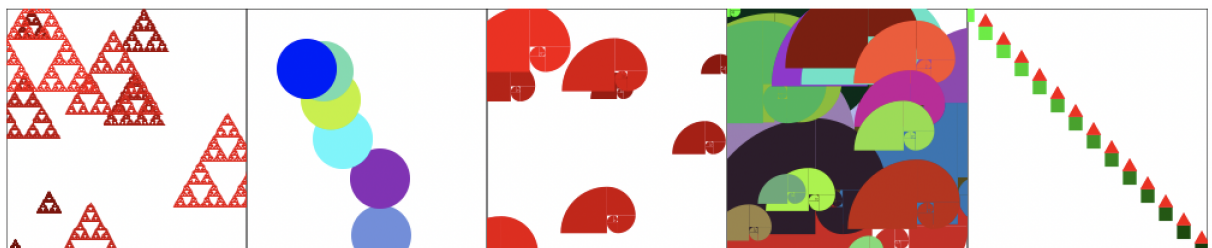
```
(cena-com-figuras-gen (make-figura 50 50 2 VERMELHO) desenha-sierpinski pequena? diminui-alt)

(cena-com-figuras-gen (make-figura 100 100 1 AZUL) desenha-bola fora-da-cena? move-dir-random)

(cena-com-figuras-gen (make-figura 50 50 2 VERMELHO) desenha-espiral pequena? diminui-alt)

(cena-com-figuras-gen (make-figura 0 0 0.5 VERDE) desenha-espiral grande? aumenta-alt)

(cena-com-figuras-gen (make-figura 0 0 0.5 VERDE) desenha-casa cor-preta? move-dir)
```



- Como a função `cena-com-figuras-gen` é muito genérica, e seu critério de fim e de avanço (movimentação da figura) são argumentos da função, não é possível fazer uma argumentação genérica sobre a terminação desta função: dependendo da chamada, ela pode terminar ou não. Argumente sobre a terminação das 4 chamadas que você realizou na questão anterior. Note que se a função `random` tiver sido usada em alguma das chamadas, é possível que a execução não termine (neste caso, argumente neste sentido na terminação).